Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes

TD : Ingénierie Dirigée par les Modèles
Transformation (génération de code) par programmation

Du modèle `Library.ecore` au code Python, SQL, Java et Markdown avec l'API EMF

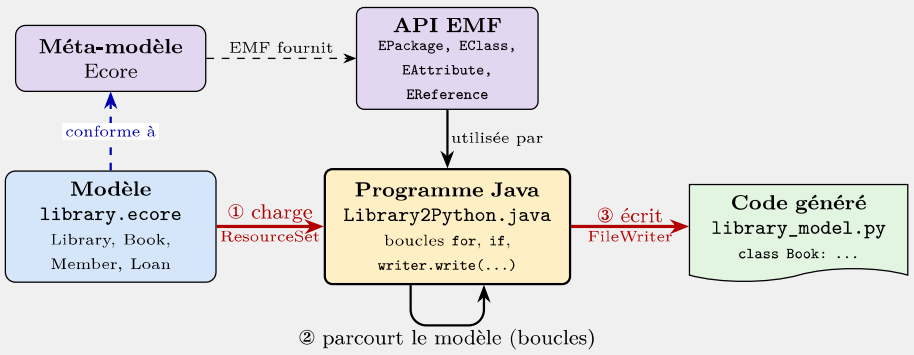
(Énoncé)

Pr.Aissam BERRAHOU
Année universitaire 2026–2027

Objectifs et consignes

1. Expliquer le principe d'une transformation **par programmation** : **charger**, **parcourir**, **écrire**.
2. Associer chaque concept du méta-modèle **Ecore** à sa méthode de l'**API EMF** (`getEClassifiers()`, `getEAttributes()`, `isMany()`...).
3. **Lire** un programme de transformation Java et **prédire** exactement le texte produit.
4. **Écrire** des générateurs (Python, SQL, Java, Markdown) et des méthodes utilitaires ; **corriger** un générateur erroné.
5. **Comparer** la programmation et le template (Acceleo).

Le TD suit le **fil rouge du cours** (Lecture 4) et du **Tutoriel 3 – Transformation par programmation** : le modèle `library.ecore`. Un second petit modèle, `garage.ecore`, permet de vérifier que les générateurs sont bien **indépendants** du modèle. Les exercices se font **sur papier** ; le TP associé permet d'exécuter les mêmes programmes dans Eclipse. Le code est en **Java 17** ; l'aide-mémoire de l'API EMF (fin du cours) est autorisé.

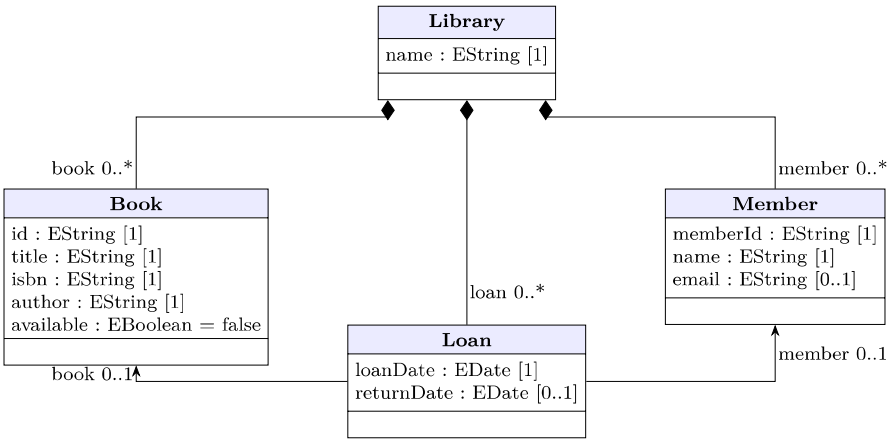


Plan du TD :

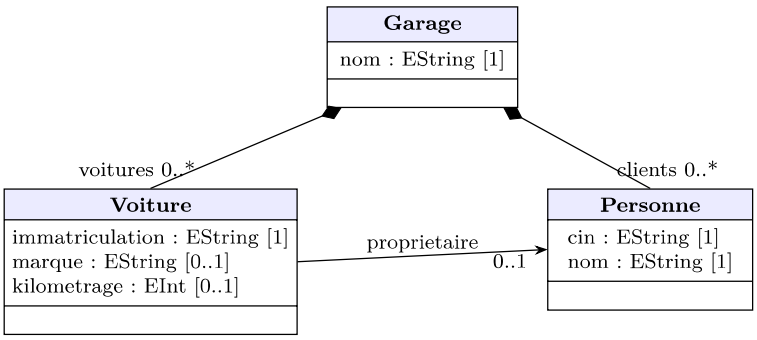
Exercice	Thème	Notion principale
1	Vrai ou faux ?	Principe de la programmation [Simple]
2	Du méta-modèle Ecore à l'API Java	Getters de l'API EMF [Simple]
3	Charger un modèle	`ResourceSet`, fabrique, erreurs [Simple]
4	Prédire la sortie d'un programme	Boucles, séparateurs [Simple]
5	Compléter le générateur Python du tutoriel	Boucles imbriquées, `isMany()` [Moyen]
6	Méthodes utilitaires	Correspondance de types, noms [Moyen]
7	Trouver et corriger les erreurs	Écriture de fichiers, `equals` [Moyen]
8	Générer le script SQL de `garage.ecore`	Clés primaires et étrangères [Moyen]
9	Documentation Markdown	Nouvelle cible, multiplicités [Avancé]
10	Programmation ou template ?	Traduire Java ↔ Acceleo [Avancé]

Rappel : les modèles utilisés

Le modèle `library.ecore` (paquetage `library`) a été dessiné avec Ecore Tools. Les attributs notés `[1]` sont obligatoires (`lowerBound = 1`) ; `[0..1]` sont facultatifs (`lowerBound = 0`). Les losanges noirs sont des **compositions** (`containment = true`).

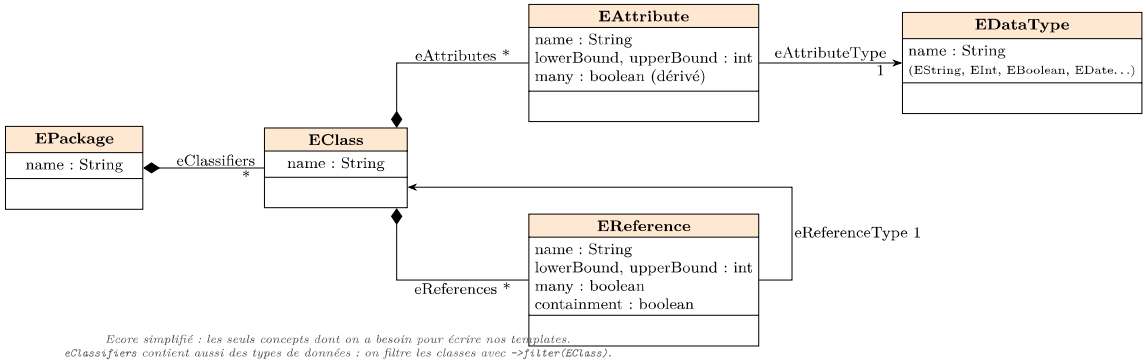


Le second modèle, `garage.ecore` (paquetage `garage`) :



Modèle `garage.ecore` (paquetage `garage`)

Rappel du méta-modèle **Ecore** simplifié et de la règle « propriété `xxx` → méthode `getXxx()` / `isXxx()` » :



Exercice 1 : Vrai ou faux ? [Simple]

Répondre par **vrai** ou **faux** et **corriger** les affirmations fausses.

A1. Une transformation par programmation ne peut produire que du texte, jamais un modèle.

A2. Pour lire un fichier `.ecore` en Java, on utilise l'API EMF (`org.eclipse.emf.ecore`).

A3. `ePackage.getEClassifiers()` renvoie uniquement les classes (`EClass`) du paquetage.

A4. La méthode qui donne la propriété `containment` d'une `EReference` est `getContainment()`.

A5. Le programme `Library2Python` doit être modifié si l'on ajoute une classe `Author` au modèle.

A6. En dehors d'Eclipse, il faut enregistrer une fabrique de ressources pour l'extension `ecore`.

A7. Avec `FileWriter`, si l'on oublie de fermer le fichier, il peut rester vide.

A8. En Java, `type == "EString"` est la bonne façon de comparer deux chaînes.

A9. Les zones protégées existent automatiquement en programmation, comme `[protected]` en Acceleo.

A10. Avec `String.join(", ", liste)`, une virgule est écrite après le dernier élément.

Exercice 2 : Du méta-modèle Ecore à l'API Java [Simple]

On a chargé `library.ecore` dans la variable `EPackage p`. Les variables suivantes désignent des éléments du modèle : `book` et `loan` (les `EClass` Book et Loan), `email` (l'`EAttribute` email de Member), `libBook` (l'`EReference` book de Library) et `loanBook` (l'`EReference` book de Loan).

a) Donner la valeur de chaque expression Java.

N°	Expression Java	Valeur
1	`p.getName()`	
2	`p.getEClassifiers().size()`	
3	`book.getEAttributes().size()`	
4	`book.getEAttributes().get(4).getName()`	
5	`book.getEAttributes().get(4)` `.getEAttributeType().getName()`	
6	`email.getLowerBound()`	
7	`libBook.isMany()` / `libBook.isContainment()`	
8	`loanBook.isMany()` / `loanBook.isContainment()`	
9	`loanBook.getEReferenceType().getName()`	
10	`loan.getEReferences().size()`	
11	`book.getEReferences().isEmpty()`	
12	`p.getNsURI()`	

b) Écrire l'expression (ou les quelques lignes) Java qui calcule :

b1. le nom de la classe cible de la référence `member` de Loan (`loan` est connu) ;

b2. vrai si l'attribut `attr` est obligatoire ;

b3. le nombre de classes du paquetage `p` (sans compter les éventuels types de données) ;

b4. la liste des noms des attributs de type `EString` d'une classe `cls`.

Exercice 3 : Charger un modèle [Simple]

a) Les quatre instructions du chargement ont été mélangées. Les remettre dans l'ordre et dire, pour chacune, ce qu'elle fait.

```
(A)  EPackage ePackage = (EPackage) resource.getContents().get(0);
(B)  ResourceSet resourceSet = new ResourceSetImpl();
(C)  Resource resource = resourceSet.getResource(URI.createFileURI("model/library.ecore"), true);
(D)  resourceSet.getResourceFactoryRegistry().getExtensionToFactoryMap()
                .put("ecore", new XMIResourceFactoryImpl());
```

b) Pourquoi faut-il écrire `(EPackage)` devant `resource.getContents().get(0)` ?

c) Pour chacune des situations suivantes, dire quelle erreur apparaît et comment la corriger :

c1. on oublie l'instruction (D) ;

c2. on écrit `"model/Library.ecore"` alors que le fichier s'appelle `library.ecore` (sous Linux ou macOS) ;

c3. on charge `model/biblio.xmi`, un fichier d'**instances** (des livres et des membres) conforme à `library.ecore` ;

c4. Eclipse souligne en rouge `import org.eclipse.emf.ecore.*;`.

Exercice 4 : Prédire la sortie d'un programme [Simple]

On exécute le programme suivant sur `garage.ecore` (`EcoreLoader.load` est la méthode de chargement vue en cours).

```
EPackage p = EcoreLoader.load("model/garage.ecore");
System.out.println(p.getName() + " : " + p.getEClassifiers().size() + " classe(s)");
for (EClassifier c : p.getEClassifiers()) {
    if (c instanceof EClass cls) {
        String line = cls.getName() + " (";
        for (EAttribute a : cls.getEAttributes()) {
            line += a.getName() + (a.getLowerBound() > 0 ? "*" : "") + ", ";
        }
        System.out.println(line + ")");
        for (EReference r : cls.getEReferences()) {
            if (!r.isContainment()) {
                System.out.println("  " + r.getName() + " -> " + r.getEReferenceType().getName());
            }
        }
    }
}
```

a) Écrire exactement ce qu'affiche la console.

b) Que signifie l'étoile `*` ? Pourquoi les références `voitures` et `clients` ne sont-elles pas affichées ?

c) L'affichage contient un défaut de séparateur. Lequel ? Réécrire la boucle des attributs avec `String.join` pour obtenir `Garage (nom*)`.

d) Qu'affiche ce second programme sur `garage.ecore` ?

```
for (EClassifier c : p.getEClassifiers()) {
    if (c instanceof EClass cls) {
        List<String> noms = new ArrayList<>();
        for (EAttribute a : cls.getEAttributes()) {
            if (a.getEAttributeType().getName().equals("EString")) noms.add(a.getName());
        }
        System.out.println(cls.getName() + " : " + String.join(" | ", noms));
    }
}
```

Exercice 5 : Compléter le générateur Python du tutoriel [Moyen]

On veut produire `<paquetage>_model.py` : une classe Python par `EClass`, un constructeur dont les paramètres sont les attributs (valeur par défaut `None`), puis une ligne d'initialisation par attribut et par référence. Compléter les trous ① à ⑥.

```
public void generatePython(EPackage ePackage, String outDir) throws IOException {
    Files.createDirectories(Paths.get(outDir));
    String fileName = outDir + "/" + ePackage.getName() + "_model.py";
    try (① .........) {
        writer.write("# -*- coding: utf-8 -*-\n");
        for (EClassifier classifier : ② .........) {
            if (③ .........) {
                writer.write("\n\nclass " + cls.getName() + ":\n");
                writer.write("    def __init__(self");
                for (EAttribute attr : ④ .........) {
                    writer.write(", " + attr.getName() + "=None");
                }
                writer.write("):\n");
                for (EAttribute attr : cls.getEAttributes()) {
                    writer.write("        self." + ⑤ ......... + "\n");
                }
                for (EReference ref : cls.getEReferences()) {
                    writer.write("        self." + ref.getName() + " = " + ⑥ ......... + "\n");
                }
            }
        }
    }
}
```

a) Compléter les trous. Pour ⑥, le tutoriel écrit simplement `"[]"` : que produit alors le programme pour la classe `Loan` ? Quel est le défaut ? Proposer une meilleure expression.

b) Écrire le texte produit pour la classe `Voiture` de `garage.ecore`.

c) Quelle classe d'un modèle ferait produire un code Python **invalide** ? Comment corriger le générateur ?

d) Pourquoi utiliser `try ( ... )` plutôt que `writer.close()` à la fin, comme dans le tutoriel ?

Exercice 6 : Méthodes utilitaires [Moyen]

Comme les queries d'Acceleo, les **méthodes utilitaires** ne produisent pas de texte : elles calculent une valeur que le générateur écrit ensuite.

a) Écrire `static String toPythonType(String typeName)` : EString → `str`, EInt → `int`, EBoolean → `bool`, EDouble → `float`, EDate → `date`, sinon `object`.

b) Écrire `static String upperFirst(String s)` qui met la première lettre en majuscule (`"title"` → `"Title"`).

c) Écrire `static String getterName(EAttribute attr)` qui renvoie `isAvailable` pour un booléen et `getTitle` sinon.

d) Écrire `static String initSignature(EClass cls)` qui renvoie la signature Python **annotée**, par exemple pour Member :
`def __init__(self, memberId: str = None, name: str = None, email: str = None):`

e) Donner `initSignature` pour `Loan` et pour `Voiture`, et les noms des getters de `Book`.

f) Pourquoi ranger ces correspondances dans des méthodes séparées ? Faire le lien avec le **PDM** de MDA.

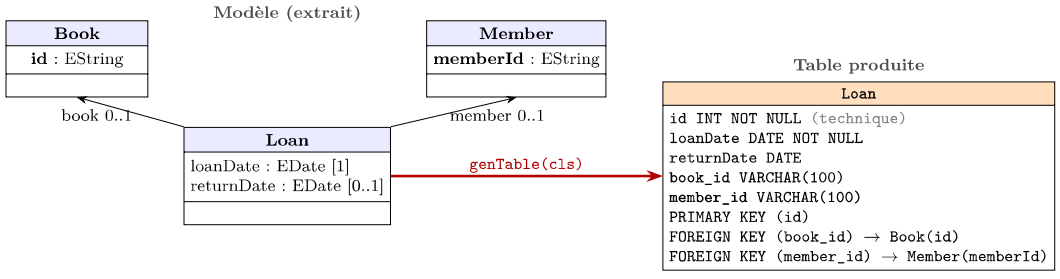
Exercice 7 : Trouver et corriger les erreurs [Moyen]

Le générateur SQL suivant contient **7 erreurs** (compilation, exécution ou logique). Les lignes sont numérotées en commentaire. Pour chacune : donner la ligne, expliquer l'erreur et la corriger. Écrire ensuite la version corrigée.

```
public void generateSQL(EPackage ePackage) throws IOException {
    FileWriter writer = new FileWriter("gen/schema.sql");                    // 1
    for (EClassifier classifier : ePackage.getEClassifiers()) {              // 2
        EClass cls = (EClass) classifier;                                     // 3
        writer.write("CREATE TABLE " + cls.getName() + " (");                 // 4
        for (EAttribute attr : cls.getEAttributes()) {                        // 5
            writer.write("    " + attr.getName() + " "
                         + toSQLType(attr.getName()) + ",\n");                // 6
        }
        if (cls.getName().equals("Book")) {                                   // 7
            writer.write("    PRIMARY KEY (id)\n");                           // 8
        }
        writer.write(");\n\n");                                               // 9
    }
}                                                                             // 10
static String toSQLType(String typeName) {
    if (typeName == "EString") return "VARCHAR(100)";                         // 11
    if (typeName == "EInt") return "INT";                                     // 12
    return "VARCHAR(255)";
}
```

Exercice 8 : Générer le script SQL de `garage.ecore` [Moyen]

Le générateur `Library2SQL` du cours applique les règles suivantes :



① pas d'attribut `id`/`...Id` ⇒ colonne technique `id INT` ② `lowerBound = 1` ⇒ `NOT NULL`
③ référence simple **non contenante** ⇒ colonne `<nom>_id` + `FOREIGN KEY` vers la clé primaire de la classe cible

a) Appliquer ces règles à `garage.ecore` : écrire le script SQL produit (même en-tête que pour `library.sql`).

b) La convention « `id` ou `...Id` » est-elle adaptée à `Personne` ? Proposer une meilleure règle en utilisant la case **ID** d'un attribut dans Ecore Tools (méthode `attr.isID()`), et réécrire `pkAttribute(EClass cls)`.

c) Le script obtenu est accepté par SQLite mais **refusé par MySQL**. Pourquoi ? Proposer deux corrections du générateur.

Exercice 9 : Documentation Markdown [Avancé]

On veut générer automatiquement la documentation d'un modèle dans un fichier `gen/<paquetage>.md`. Pour `Loan`, on attend :

```
## Loan

| Propriété | Type | Multiplicité |
|---|---|---|
| loanDate | EDate | 1 |
| returnDate | EDate | 0..1 |
| book | → Book | 0..1 |
| member | → Member | 0..1 |
```

Une composition est notée ♦ `Book`, une référence simple → `Book`. Le fichier commence par `# Modèle library`.

a) Écrire `static String multiplicity(ETypedElement e)` qui renvoie `1`, `0..1`, `0..*` ou `1..*`. (`ETypedElement` est la super-interface commune d'`EAttribute` et d'`EReference`.)

b) Écrire `generateMarkdown(EPackage ePackage, String outDir)` avec un `StringBuilder`.

c) Écrire la section produite pour `Garage`.

Exercice 10 : Programmation ou template ? [Avancé]

a) Traduire ce template Acceleo en Java (on suppose un `Writer writer` déjà ouvert). Qu'écrit-il pour `library.ecore` ? pour `garage.ecore` ?

```
[for (cls : EClass | root.eClassifiers->filter(EClass))]
[if (cls.eReferences->exists(r | r.containment))]
# [cls.name/] est un conteneur
[/if]
[/for]
```

b) Traduire en Acceleo cette ligne Java, qui écrit la liste des attributs d'une classe :

```
List<String> noms = new ArrayList<>();
for (EAttribute a : cls.getEAttributes()) noms.add(a.getName());
writer.write(cls.getName() + "(" + String.join(", ", noms) + ")\n");
```

c) Pour chacun des besoins suivants, quelle approche choisiriez-vous ? Justifier en une phrase.

c1. Générer une page HTML très régulière par classe (50 classes).

c2. Générer un rapport qui trie les classes selon le nombre de classes qui les référencent.

c3. Intégrer la génération dans une application web (bouton « Générer le SQL »).